Customer Sync Pipeline

Code:

```
import os
import logging
from datetime import datetime, timedelta

import pandas as pd

# from airflow.decorators import dag
from airflow import DAG
from airflow.operators.python import PythonOperator, BranchPythonOperator
from airflow.providers.snowflake.hooks.snowflake import SnowflakeHook
from airflow.exceptions import AirflowException


logger = logging.getLogger(__name__)


def alert_on_failure(context):
    ti = context["task_instance"]
    dag_id = ti.dag_id
    task_id = ti.task_id
    print(f"ALERT: Task {task_id} in DAG {dag_id} has failed. Check the
logs.")

def validate_file():

    try:

        file_path = "data/customers_data.csv"

        if not os.path.exists(file_path):
            raise AirflowException(
                "customers_data.csv not found"
            )

        logger.info(
            "CSV file found — proceeding with load."
        )

        return file_path

    except AirflowException:
        raise

    except Exception as e:
```

```
        logger.error(f"Error in validate_file: {str(e)}")
        raise AirflowException(f"validate_file failed: {str(e)}")


def create_and_load():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        # Create table
        create_table_sql = """
        CREATE TABLE IF NOT EXISTS RAW_CUSTOMERS (
            customer_id INT,
            name VARCHAR,
            email VARCHAR,
            city VARCHAR,
            signup_date DATE,
            plan VARCHAR,
            loaded_at TIMESTAMP
        );
        """

        hook.run(create_table_sql)

        logger.info("RAW_CUSTOMERS table ready.")

        # Idempotency delete
        hook.run("""
            DELETE FROM RAW_CUSTOMERS
            WHERE loaded_at::DATE = CURRENT_DATE();
        """)

        logger.info("Cleared today's existing RAW_CUSTOMERS rows.")

        # Read CSV and insert rows
        # Read CSV and insert rows
        file_path = "data/customers_data.csv"
        df = pd.read_csv(file_path)

        values_list = []

        for _, row in df.iterrows():

            name = str(row['name']).replace("'", "''")
            email = str(row['email']).replace("'", "''")
            city = str(row['city']).replace("'", "''")
```

```
            plan = str(row['plan']).replace("'", "''")

            values_list.append(f"""
                (
                    {row['customer_id']},
                    '{name}',
                    '{email}',
                    '{city}',
                    '{row['signup_date']}',
                    '{plan}',
                    CURRENT_TIMESTAMP()
                )
            """)

        # Insert in batches of 100
        batch_size = 100

        for i in range(0, len(values_list), batch_size):

            batch = values_list[i: i + batch_size]

            insert_sql = f"""
                INSERT INTO RAW_CUSTOMERS (
                    customer_id,
                    name,
                    email,
                    city,
                    signup_date,
                    plan,
                    loaded_at
                )
                VALUES {', '.join(batch)};
            """

            hook.run(insert_sql)

            logger.info(
                f"Inserted batch {i // batch_size + 1} "
                f"({len(batch)} rows)."
            )

        logger.info(f"Loaded {len(df)} rows into RAW_CUSTOMERS.")

    except AirflowException:
        raise

    except Exception as e:
        logger.error(f"Error in create_and_load: {str(e)}")
```

```
        raise AirflowException(f"create_and_load failed: {str(e)}")

def enrich_customers():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        # Create enriched table
        create_enriched_sql = """
        CREATE TABLE IF NOT EXISTS ENRICHED_CUSTOMERS (
            customer_id INT,
            name VARCHAR,
            email VARCHAR,
            city VARCHAR,
            signup_date DATE,
            plan VARCHAR,
            customer_tier VARCHAR,
            days_since_signup INT,
            loaded_at TIMESTAMP
        );
        """

        hook.run(create_enriched_sql)

        logger.info("ENRICHED_CUSTOMERS table ready.")

        # Idempotency delete
        hook.run("""
            DELETE FROM ENRICHED_CUSTOMERS
            WHERE loaded_at::DATE = CURRENT_DATE();
        """)

        logger.info("Cleared today's existing ENRICHED_CUSTOMERS rows.")

        # Insert enriched data
        insert_enriched_sql = """
        INSERT INTO ENRICHED_CUSTOMERS (
            customer_id,
            name,
            email,
            city,
            signup_date,
            plan,
            customer_tier,
            days_since_signup,
```

```
            loaded_at
        )
        SELECT
            customer_id,
            name,
            email,
            city,
            signup_date,
            plan,
            CASE WHEN plan = 'premium' THEN 'Premium Tier' ELSE 'Basic Tier'
END,
            DATEDIFF('day', signup_date, CURRENT_DATE()),
            CURRENT_TIMESTAMP()
        FROM RAW_CUSTOMERS
        WHERE loaded_at::DATE = CURRENT_DATE();
        """

        hook.run(insert_enriched_sql)

        # Log enriched row count
        count = hook.get_first("""
            SELECT COUNT(*)
            FROM ENRICHED_CUSTOMERS
            WHERE loaded_at::DATE = CURRENT_DATE();
        """)[0]

        logger.info(f"{count} enriched rows created in ENRICHED_CUSTOMERS.")

    except AirflowException:
        raise

    except Exception as e:
        logger.error(f"Error in enrich_customers: {str(e)}")
        raise AirflowException(f"enrich_customers failed: {str(e)}")


def quality_check():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        # Check 1 — NULL check
        null_count = hook.get_first("""
            SELECT COUNT(*)
            FROM ENRICHED_CUSTOMERS
            WHERE (name IS NULL OR email IS NULL)
```

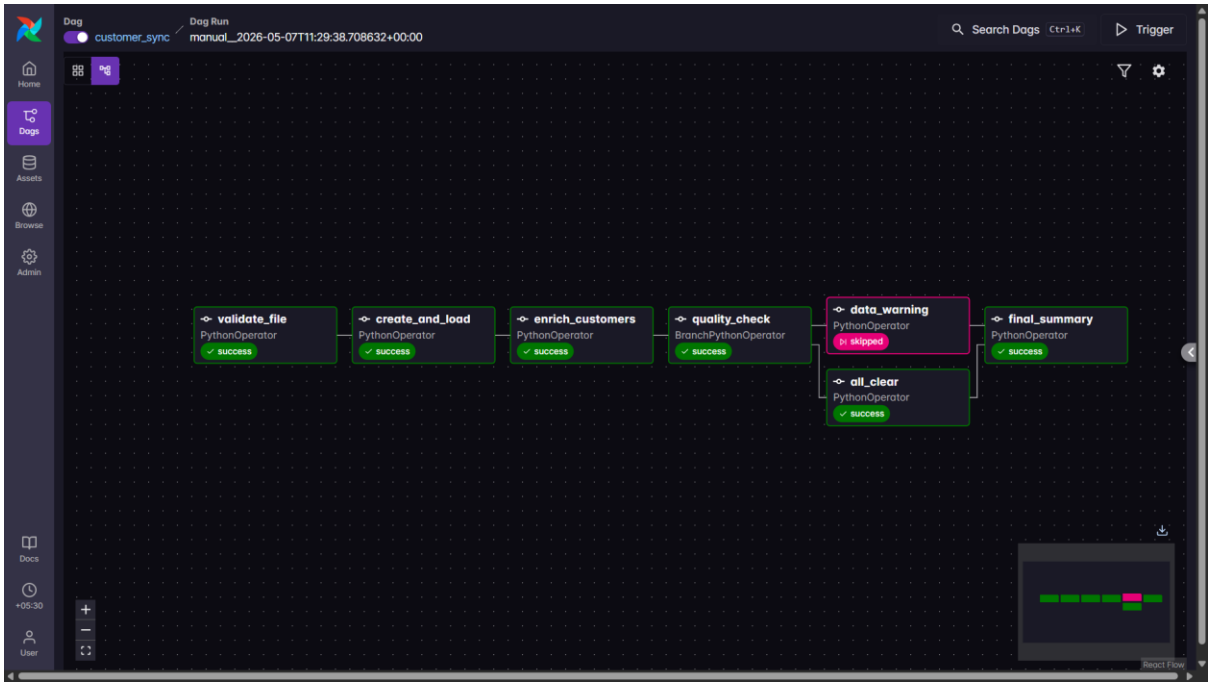
```
            AND loaded_at::DATE = CURRENT_DATE();
        """)[0]

        logger.info(f"NULL check result: {null_count} rows with nulls.")

        # Check 2 — Duplicate check
        duplicate_count = hook.get_first("""
            SELECT COUNT(*) - COUNT(DISTINCT customer_id)
            FROM ENRICHED_CUSTOMERS
            WHERE loaded_at::DATE = CURRENT_DATE();
        """)[0]

        logger.info(f"Duplicate check result: {duplicate_count} duplicate
customer_ids.")

        if null_count == 0 and duplicate_count == 0:
            logger.info("Both checks passed — routing to all_clear.")
            return "all_clear"
        else:
            logger.warning("One or more checks failed — routing to
data_warning.")
            return "data_warning"

    except AirflowException:
        raise

    except Exception as e:
        logger.error(f"Error in quality_check: {str(e)}")
        raise AirflowException(f"quality_check failed: {str(e)}")


def all_clear():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        logger.info(
            "Quality checks passed — customer data is clean."
        )

        count = hook.get_first("""
            SELECT COUNT(*)
            FROM ENRICHED_CUSTOMERS
            WHERE loaded_at::DATE = CURRENT_DATE();
        """)[0]
```

```
        logger.info(f"Total enriched rows for today: {count}")

    except AirflowException:
        raise

    except Exception as e:
        logger.error(f"Error in all_clear: {str(e)}")
        raise AirflowException(f"all_clear failed: {str(e)}")


def data_warning():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        null_count = hook.get_first("""
            SELECT COUNT(*)
            FROM ENRICHED_CUSTOMERS
            WHERE (name IS NULL OR email IS NULL)
            AND loaded_at::DATE = CURRENT_DATE();
        """)[0]

        duplicate_count = hook.get_first("""
            SELECT COUNT(*) - COUNT(DISTINCT customer_id)
            FROM ENRICHED_CUSTOMERS
            WHERE loaded_at::DATE = CURRENT_DATE();
        """)[0]

        logger.warning(
            "WARNING: Data quality issues detected in customer load."
        )

        if null_count > 0:
            logger.warning(f"NULL check FAILED — {null_count} rows with null
name or email.")

        if duplicate_count > 0:
            logger.warning(f"Duplicate check FAILED — {duplicate_count}
duplicate customer_ids.")

    except AirflowException:
        raise

    except Exception as e:
        logger.error(f"Error in data_warning: {str(e)}")
        raise AirflowException(f"data_warning failed: {str(e)}")
```

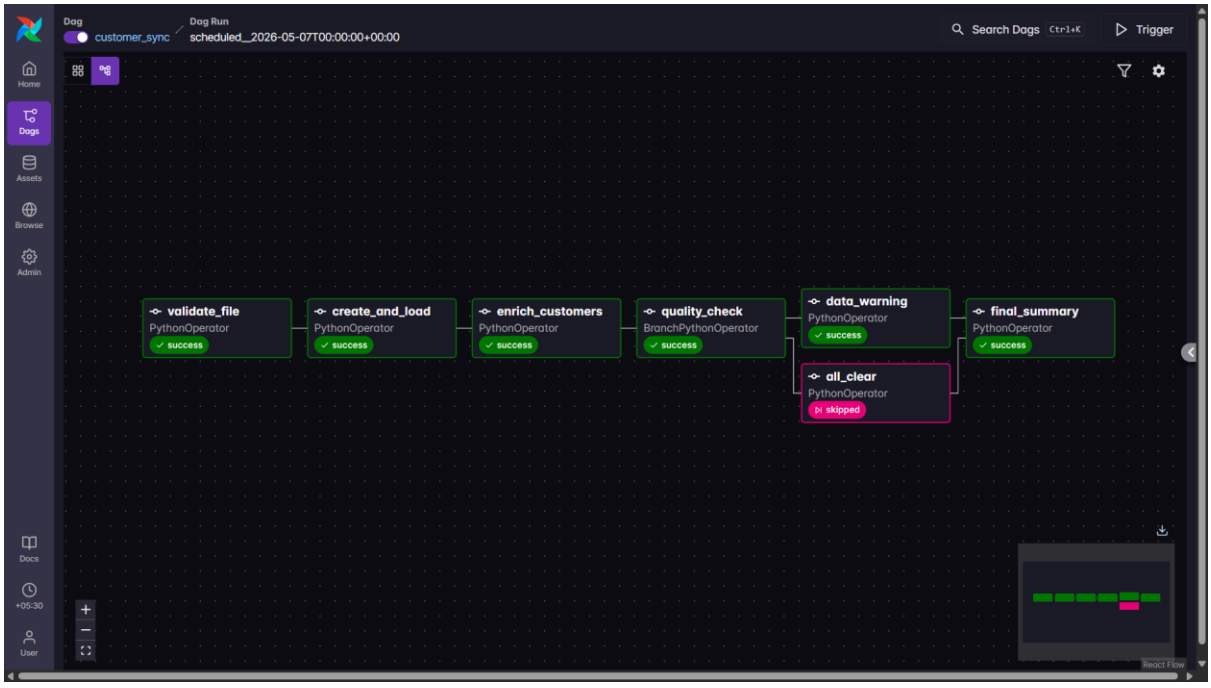
```
def final_summary():

    try:

        hook = SnowflakeHook(
            snowflake_conn_id='snowflake_default'
        )

        summary = hook.get_records("""
            SELECT
                COUNT(*) AS total_customers,
                SUM(CASE WHEN customer_tier = 'Premium Tier' THEN 1 ELSE 0
END) AS premium_count,
                SUM(CASE WHEN customer_tier = 'Basic Tier' THEN 1 ELSE 0 END)
AS basic_count,
                ROUND(AVG(days_since_signup), 1) AS avg_days_since_signup,
                MODE(city) AS top_city
            FROM ENRICHED_CUSTOMERS
            WHERE loaded_at::DATE = CURRENT_DATE();
        """)

        row = summary[0]

        logger.info(f"Total Customers: {row[0]}")
        logger.info(f"Premium Tier: {row[1]}")
        logger.info(f"Basic Tier: {row[2]}")
        logger.info(f"Average Days Since Signup: {row[3]}")
        logger.info(f"City with Most Customers: {row[4]}")

    except AirflowException:
        raise

    except Exception as e:
        logger.error(f"Error in final_summary: {str(e)}")
        raise AirflowException(f"final_summary failed: {str(e)}")
default_args = {
    'owner': 'airflow',
    'retries': 1,
    'retry_delay': timedelta(minutes=5),
    'on_failure_callback': alert_on_failure
}


with DAG(
    dag_id='customer_sync',
    schedule='@daily',
    start_date=datetime(2023, 1, 1),
    catchup=False,
```

```
    default_args=default_args,
    on_failure_callback=alert_on_failure
) as dag:

    validate_file_task = PythonOperator(
        task_id='validate_file',
        python_callable=validate_file
    )

    create_and_load_task = PythonOperator(
        task_id='create_and_load',
        python_callable=create_and_load
    )

    enrich_customers_task = PythonOperator(
        task_id='enrich_customers',
        python_callable=enrich_customers
    )

    quality_check_task = BranchPythonOperator(
        task_id='quality_check',
        python_callable=quality_check
    )

    all_clear_task = PythonOperator(
        task_id='all_clear',
        python_callable=all_clear
    )

    data_warning_task = PythonOperator(
        task_id='data_warning',
        python_callable=data_warning
    )

    final_summary_task = PythonOperator(
        task_id='final_summary',
        python_callable=final_summary,
        trigger_rule='none_failed_min_one_success'
    )

    validate_file_task >> create_and_load_task >> enrich_customers_task >>
quality_check_task
    quality_check_task >> [all_clear_task, data_warning_task] >>
final_summary_task
```

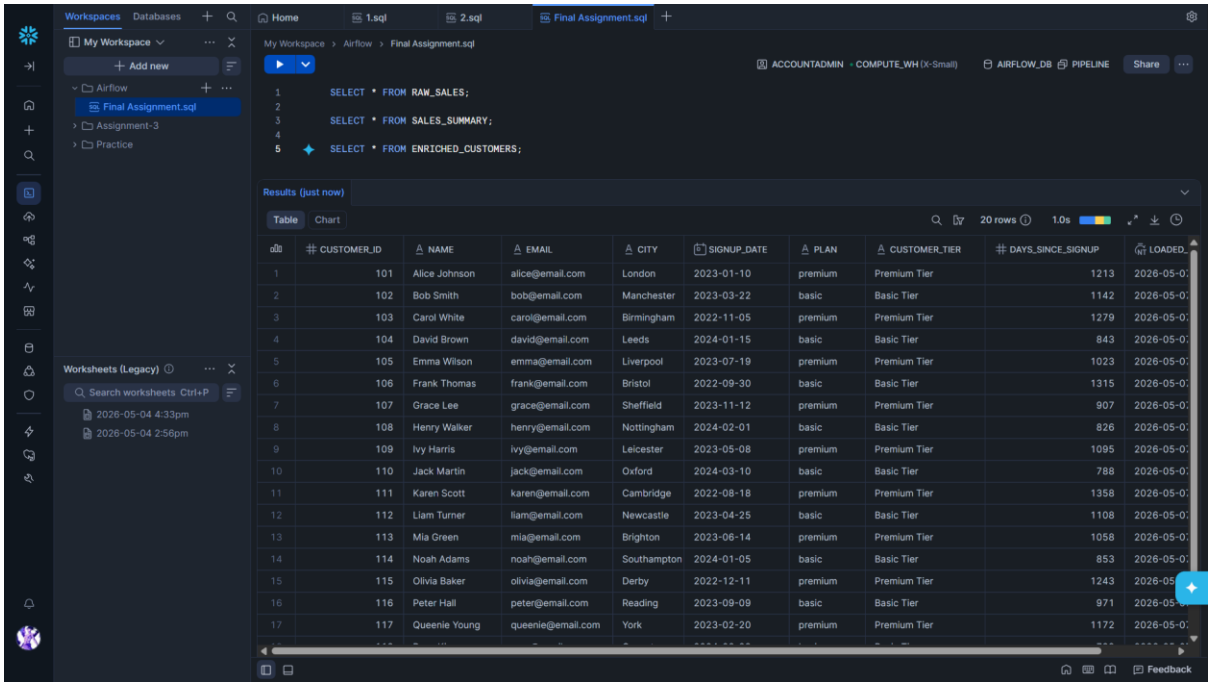
1. Graph View — Run A: all_clear GREEN, data_warning pink (skipped), final_summary GREEN



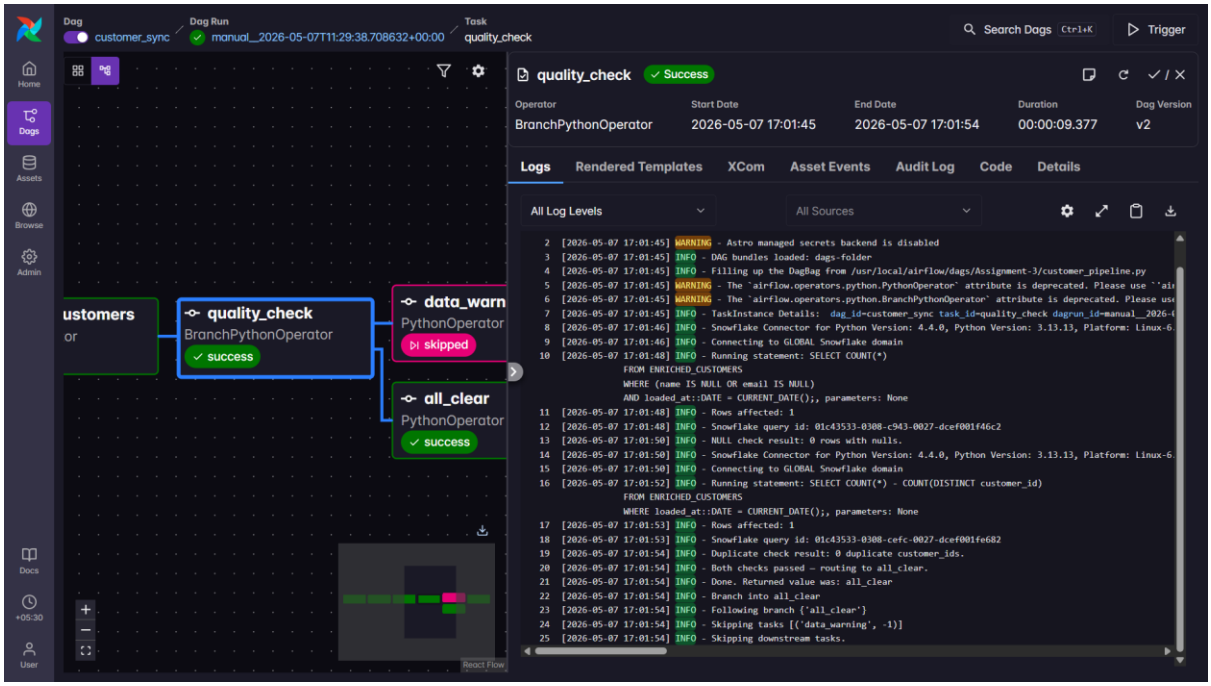
2. Graph View — Run B (add a bad row to trigger warning): data_warning GREEN, all_clear pink



3. Snowflake Worksheet — SELECT * FROM ENRICHED_CUSTOMERS showing customer_tier and days_since_signup



4. Task Log of quality_check — both check values logged before the branch decision



5. Task Log of final_summary — tier breakdown report with averages